3D Slicer Physical Integration & End-to-End Testing Plan

1. Executive Summary & Integration Architecture

The **Surgical Physics Bridge** connects an external 1000 Hz PyBullet physics engine and GPU ultrasound simulator with **3D Slicer (v5.0+)** via **OpenIGTLink (pyigtl)** binary TCP sockets. While standalone unit tests (`pytest`) verify kinematic transformations and message serialization in memory, actual clinical deployment requires compiling the C++ Loadable Module (`SlicerSurgicalBridge`) and running it inside a live 3D Slicer desktop session.

2. Gap Analysis: Isolation vs. Live Slicer Runtime

Subsystem	Verified in Isolation (Unit Tests)	Requires Live 3D Slicer Testing	Potential Failure Modes
OpenIGTLink Transport	`pyigtl` message packing/unpacking in memory	`vtkMRMLIGTLConnectorNode` socket client/server handshake	Port blocking, endianness mismatch, socket timeout during heavy rendering
Transform Updates	4x4 matrix conversion (`pybullet_to_slicer_ras_matrix`)	Real-time hierarchical MRML transform node propagation	Coordinate frame flip (RAS vs LPS), visual stutter if framerate drops
Live B-Mode Ultrasound	Synthetic 2D NumPy array generation (< 2< 2 ms)	`vtkMRMLVectorVolumeNode` or `vtkMRMLScalarVolumeNode` live slice display	Slicer slice viewer pipeline stall if `Modified()` called incorrectly
Point Cloud Fusion	Voxel grid spatial accumulation	`vtkPolyData` in-place memory buffer update (`ptsData->SetComponent`)	Memory leaks, segfaults if point buffer count changes dynamically
Contact Force Arrows	Nx7 NumPy array generation	`vtkLine` cell generation and dynamic scaling in 3D viewport	Invisible force glyphs due to millimeter vs meter scale factors
Virtual Fixtures	JSON generation from mock vertex arrays	Extraction of active Slicer model nodes and PyBullet GJK hull collision	Malformed polygon winding order, concave mesh collision failure
Interactive ROI Targeting	Hardcoded target point IK resolution	Clinician clicks via `vtkMRMLMarkupsFiducialNode` → → `StringMessage: CMD`	Coordinate frame offset between clicked fiducial and robot base

3. Step-by-Step 3D Slicer Build & Module Compilation

The module `SlicerSurgicalBridge` is a native C++ loadable extension. It must be built against a matching 3D Slicer build tree or Slicer SDK installation.

3.1 Prerequisites

1. **3D Slicer Installation**: Slicer 5.0.3, 5.2.2, or 5.6.2 (Stable Release).
2. **Slicer Extension Dependencies**: Install `OpenIGTLinkIF` via Slicer Extension Manager (`View` → → `Extension Manager`).
3. **Compiler Toolchain**:

- Linux: GCC 9+ / Clang 10+ matching the ABI of the Slicer package.
- CMake: $\geq 3.16.3$≥ 3.16.3.

3.2 Compilation Protocol

```
bash
# 1. Set Slicer Directory
export SLICER_DIR=/path/to/Slicer-5.6.2-linux-amd64
export SLICER_BUILD_DIR=/path/to/Slicer-SuperBuild/Slicer-build  # If building from
source

# 2. Configure build directory
cd /home/rishang/Desktop/Insti_Stuff/MTP
mkdir -p SlicerSurgicalBridge-build && cd SlicerSurgicalBridge-build

# 3. Configure via CMake
cmake -DCMAKE_BUILD_TYPE:STRING=Release \
      -DSlicer_DIR:PATH=${SLICER_BUILD_DIR} \
      -DSurgicalBridge_SUPERBUILD:BOOL=OFF \
      ../SlicerSurgicalBridge

# 4. Compile the Loadable Module
make -j$(nproc)
```

3.3 Loading the Module in 3D Slicer

1. Launch 3D Slicer:

   ```
   bash
   ./Slicer
   ```

2. Open **Settings**: `Edit` → → `Application Settings` → → `Modules`.
3. In **Additional module paths**, click **Add** and select:
   `/home/rishang/Desktop/Insti_Stuff/MTP/SlicerSurgicalBridge-build/lib/Slicer-5.6/qt-loadable-modules`
4. Restart 3D Slicer. The module **SurgicalBridge** will appear under the `Simulation` category.

4. Live Verification Checklist & Test Matrix

Execute the tests in sequential order. Each test must satisfy the explicit pass criterion before advancing.

Test Suite Specifications

Test ID	Objective	Procedure	Expected Outcome / Pass Criterion
TEST-01	Socket Connection	1. Start `./.venv/bin/python surgical_physics_bridge/src/unified_server.py --port 18944` 2. In Slicer `SurgicalBridge` widget, click **Connect to Physics Server**.	Connector node status changes to `CONNECTED` (green). Server console prints: `[+] Slicer client connected`. Round-trip latency < 2< 2 ms.
TEST-02	Coordinate Alignment	Observe `UltrasoundProbe` transform node in Slicer 3D view while robot moves in lateral sweep.	Probe model in Slicer moves synchronously with PyBullet simulation. Probe face points along $-X$−X into lateral flank. No 90° 90∘ pitch/yaw axis flips.
TEST-03	Live B-Mode Streaming	Observe Slicer Slice Viewer (Red or Yellow) assigned to `US_Image` image node.	Live 2D grayscale ultrasound frame updates at 30 FPS. Cortical bone hyperechoic reflection and acoustic shadows clearly visible without UI lag.
TEST-04	Point Cloud Mutation	Select a blank Model node in `Target Model` combo box. Let lateral scan run for 30 steps.	Point cloud model dynamically populates with green bone surface points. Memory usage of `SlicerApp-real` remains flat (zero garbage collection churn).
TEST-05	Force Vector Glyphs	Select a Model node in `Force Model` combo box. Apply probe contact.	Red 3D line vectors extend outward from contact points. Arrow length scales proportionally with normal force ($10.0 \times F_N$ 10.0 × FN mm).
TEST-06	Virtual Fixtures	1. Load spine model in Slicer. 2. Select model in `VF Model` combo box and click **Send Virtual Fixture**.	Server terminal prints: `[+] Virtual Fixture collision mesh updated in PyBullet`. Robot probe repels from spine canal boundary upon collision.
TEST-07	Interactive Fiducial ROI	1. Place a Markups Fiducial point on the lateral lumbar facet. 2. Click **Send ROI Target** in Slicer widget.	Server receives `{"roi_target": [x, y, z]}`, converts Slicer RAS to PyBullet world frame, and solves 7-DoF IK to position probe on flank.

5. Wiring Interactive Fiducial ROI Markups to Closed-Loop IK

In , we already implemented the fiducial reader:

```
cpp
void qSlicerSurgicalBridgeModuleWidget::onSendROITargetClicked() {
  Q_D(qSlicerSurgicalBridgeModuleWidget);
  if (!d->logic()) return;

  vtkMRMLNode* node = d->roiFiducialSelector->currentNode();
  if (auto markupsNode = vtkMRMLMarkupsFiducialNode::SafeDownCast(node)) {
    if (markupsNode->GetNumberOfControlPoints() > 0) {
      double pos[3];
      markupsNode->GetNthControlPointPosition(0, pos);
      d->logic()->SendROITarget(pos[0], pos[1], pos[2]);
      return;
    }
  }
}
```

Server-Side Frame Transformation Logic

When Slicer transmits RAS coordinates $[X_{\text{RAS}}, Y_{\text{RAS}}, Z_{\text{RAS}}]^\top$ [XRAS, YRAS, ZRAS]⊤, converts them to the PyBullet world frame using the validated coordinate utility:

$$p_{\text{PB}} = R_{\text{conv}}^{-1}\left(\frac{p_{\text{RAS}}}{1000.0} - t_{\text{conv}}\right)$$

pPB = Rconv−1 (1000.0pRAS − tconv)

Once in PyBullet space, the Kinova 7-DoF Damped Least Squares IK solver solves:

$$\Delta q = J^\top \left(JJ^\top + \lambda^2 I\right)^{-1}\left(x_{\text{target}} - x_{\text{current}}\right)$$

Δq = J⊤(JJ⊤ + λ2I)−1(xtarget − xcurrent)

This positions the probe at the lateral flank skin normal to the clicked vertebra facet.

6. Known Risks & Troubleshooting Guide

1. **Qt5 vs Qt6 Compatibility**:
    - Slicer 5.0–5.6 builds with Qt 5.15. If your host system has Qt 6 default headers, CMake will throw `qMRMLNodeComboBox` moc errors. Ensure `CMAKE_PREFIX_PATH` strictly points to Slicer's bundled Qt5.
2. **OpenIGTLink Device Name Sensitivity**:

- Slicer's `OpenIGTLinkIF` routes incoming packets based on exact `device_name` strings. Ensure server names match:
  - Transforms: `UltrasoundProbe`, `base_link`, `forearm_link`
  - Ultrasound: `US_Image`
  - Points: `ReconstructedBone`
  - Mesh: `DeformedAnatomy`
  - Forces: `ContactForces`
  - Command: `CMD`

3. **Headless Linux Workstation Display**:
   - If Slicer runs on a remote compute server without physical display, launch with `virtualgl` or `vglrun`:

```
bash
xvfb-run -s "-screen 0 1920x1080x24" ./Slicer --no-splash
```